CSEN 11 Advanced Programming

Control Structures

Control Structures

Sequence

Selection

Repetition

1. Sequence - Structured Programming

C is a structured (procedural) programming language that also gives low-level control when needed

Execution follows a clear sequence

- C programs run **one statement at a time**, in the order they are written, unless control is explicitly changed.

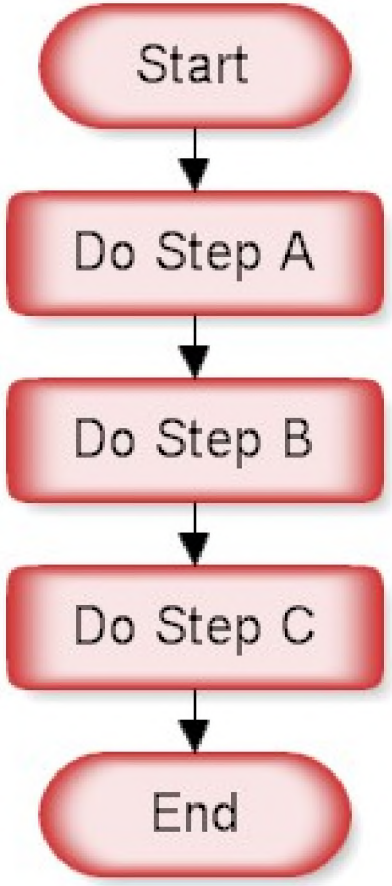
Control flow is explicit and predictable

- Any deviation from sequence happens only through well-defined constructs such as if, for, and while.

No hidden or implicit execution order

Sequence - Example

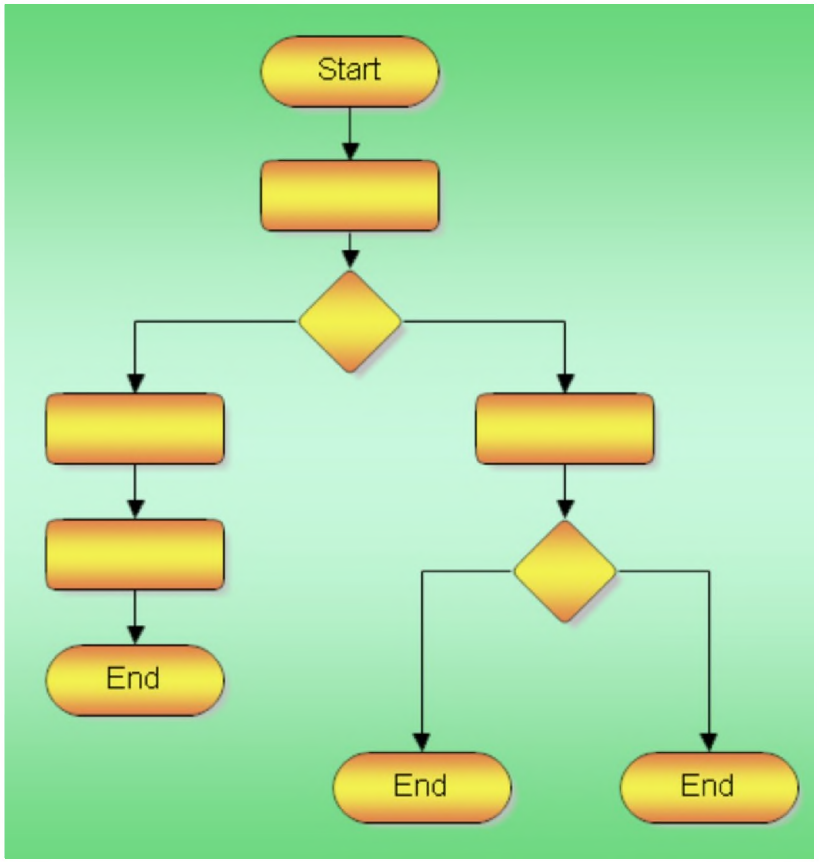
```
…
int m, x, y;
scanf ("%d%d", &x, &y);
m = x * y;
printf ("Multiplication is %d\n", m);
…
```



2. Selection Statements

Allows the program to **choose one path of execution based on a condition**

- if
- if else
- if else if
- switch

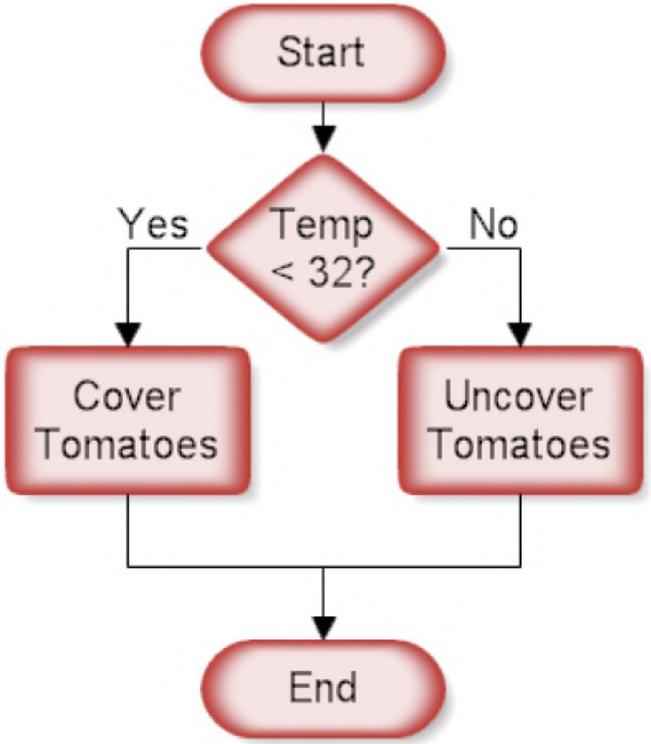


if statement - Examples

Executes a block of code **only when a condition is true**.

```
if (x > 0)   //single statement
        k++;
```

```
if (x > 0)   //more than one statement – use block
{
    k++;
        j--;
}
```



if - else statement

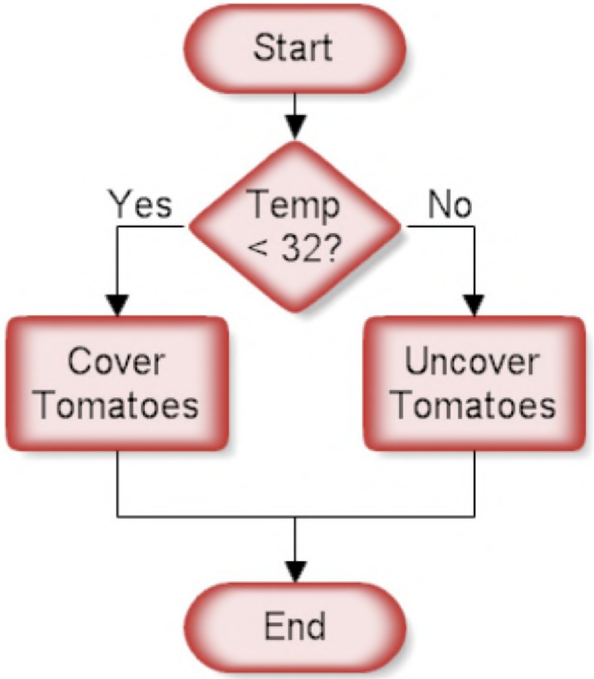
```
if (condition)
        statement 1;
else
        statement 2;
```

```
if (condition)
{
        statement 1
```

//block scope

```
statement 2
```

if–else allows a program to choose **exactly one of two code paths** based on whether a condition i



if - else - if statement

```
if (condition1)
    statement 1;
else if (condition2)
    statement 2;
else if (condition3)
    statement 3;
else
    statement 4;
```

else if: Tests an **additional condition** if and only if the previous if or else if condition is false

else: Executes a block of code when none of the preceding conditions are true

If-else-if construct

There is no elseif in C.

```
if (grade >= 90)
   printf("A\n");
else
   if (grade >= 80)
      printf("B\n");
   else
      if (grade >= 70)
         printf("C\n");
      else
         if (grade >= 60)
            printf("D\n");
         else
            printf("F\n");
```

```
if (grade >= 90)
   printf("A\n");
else if (grade >= 80
   printf("B\n");
else if (grade >= 70)
   printf("C\n");
else if (grade >= 60)
   printf("D\n");
else
   printf("F\n");
```

- Once a condition is met, the rest of the statements are skipped

nested if-else

Nested if-else places one conditional statement inside another to make decisions in multiple levels or stages.

```
if (x > y)
        if (y < z)
                a++;
        else
                b++;
else
        c++;
```

Practice!

```
int x=10, y=9, z=8, a=0, b=0, c=0;

if (x > y)
    if (y < z)
        a++;
    else
        b++;
else
    c++;
```

What are the values of a, b and c?

Practice!

```
int x=10, y=9, z=8, a=0, b=0, c=0;

if (x > y)
    if (y < z)
        a++;
    else
        b++;
else
    c++;
```

b = 1 , a = c = 0

What are the values of a, b and c?

Dangling Else

```
if (x < y)
   if (x < z)
      printf("Hello\n");
else
   printf("Goodbye\n);
```

Note: the compiler matches an else with the closest unmatched if

The above will be treated as

```
if (x < y)
   if (x < z)
      printf("Hello\n");
   else
      printf("Goodbye\n);
```

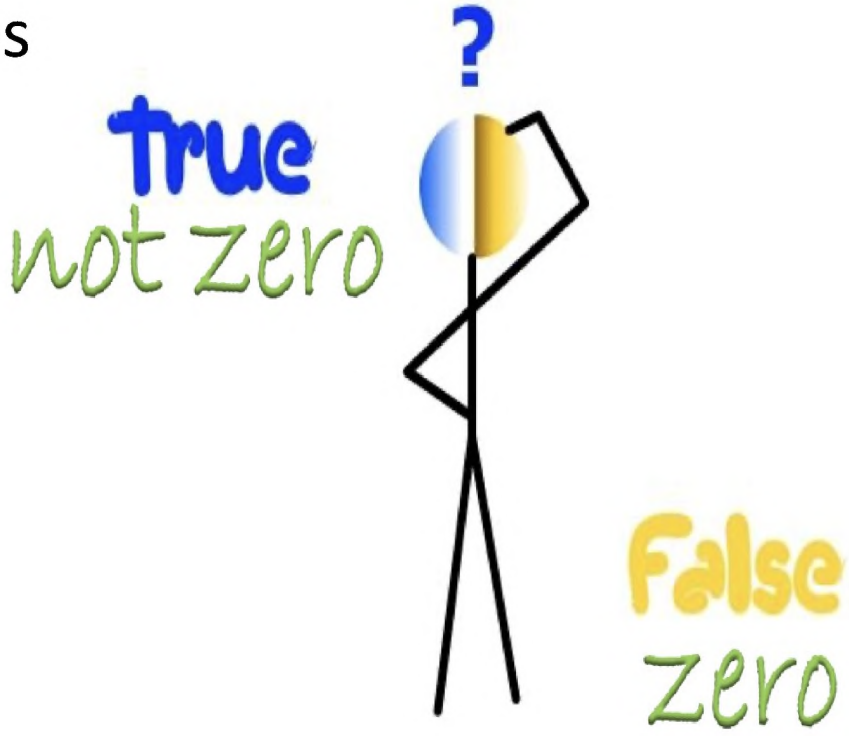
If the else is to match the outer if, use braces

```
if (x < y)
{
   if (x < z)
      printf("Hello\n");
}
else
   printf("Goodbye\n);
```

Boolean (or Conditional) Expressions

C uses integer to represent boolen values

- result is 0 or 1
  - 1 (anything non-zero) is used as true
  - 0 is used as false
- use relational and logical operators



Relational Operators

==	equality
!=	non equality
<	less than
>	greater than
<=	less than equal to
>=	greater than equal to

Equality vs Assignment

- `if (n = 0)` is not a syntax error in C.
  - The expression, *n = 0*, assigns zero to n and the value of the expression is 0. Zero is interpreted as false, and the false branch of the if statement will be taken.
- `if (n = 5)` is not a syntax error in C.
  - The expression assigns 5 to n. 5 is interpreted as true, and the true branch of the if statement will be taken.
- Remember to use the == operator to test for equality.

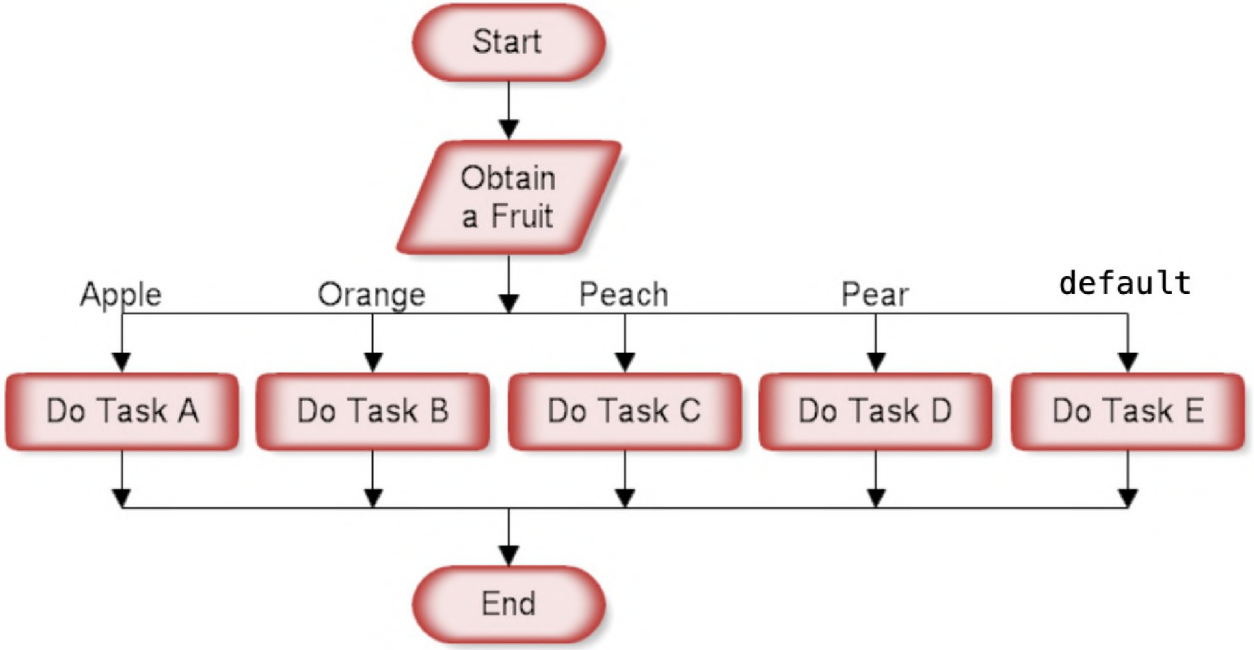
For example, use `if (0 == n)`

Logical Operators

!	not
&&	and
||	or

Switch Statement

```
switch (expression)
{
    case constant:
        statement(s);
        break;
    case constant:
        statement(s);
        break;
    default:        // default is optional
        statement(s);
```



Switch Statement

- Expression must be of type integer or character
- The keyword case must be followed by a constant
- break statement is required unless you want all subsequent statements to be executed.

Practice!

- Convert the following if/else statement to a switch statement:

```
if (choice == 1  || choice == 2)
    printf ("First Choice\n");
else if (choice == 3 || choice == 4)
    printf ("Second Choice\n");
else if (choice == 5)
        printf ("Third Choice\n");
else
        printf ("Default Choice\n");
```

3. Repetition

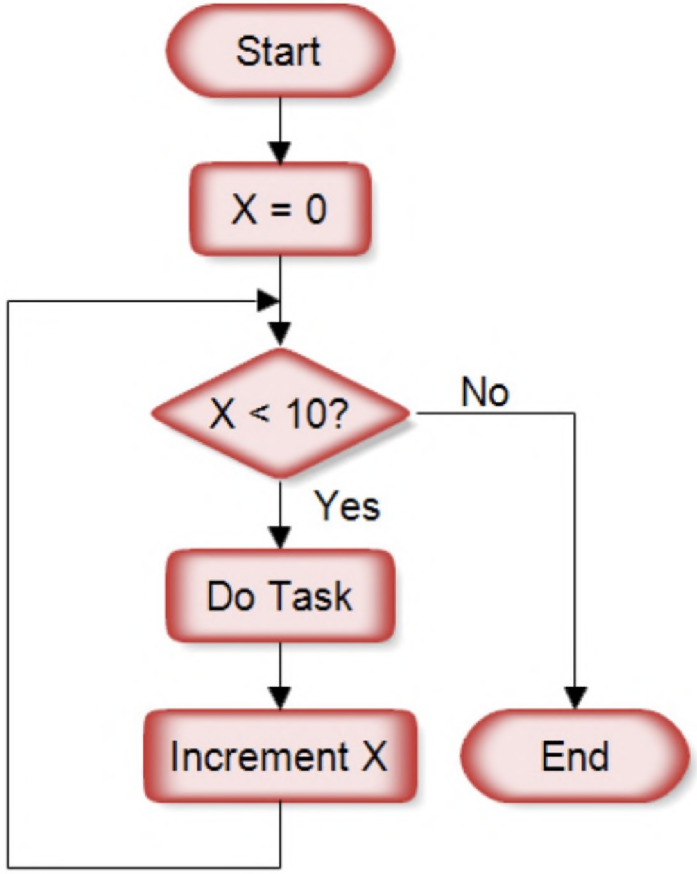
Different kinds of Loops

- while statement
- do while statement
- for statement

for statement

```
for (initialization; condition; update)
        statement;

for (initialization; condition; update)
{
        statement 1;
        ...
        statement n;
}
```



Examples

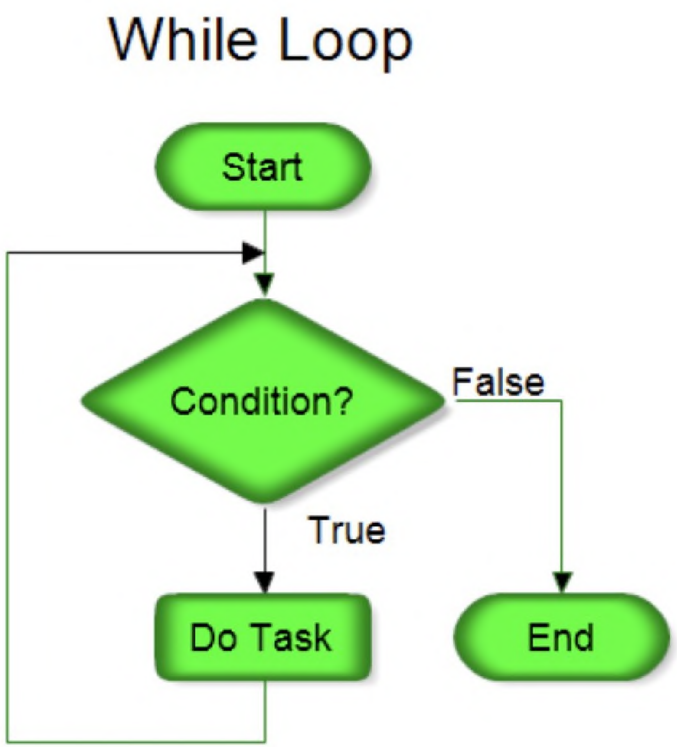
```
int sum = 0;
int i;
for (i = 1; i < 10; i += 2)
    sum = sum + i;
```

```
int fact = 1;
int n;
for (n = 5; n > 1; n--)
    fact = fact * n;
```

while statement

while (*condition*)

statement;



while (*condition*)

{

Examples

What are the values of **sum** and **i** after the loop?

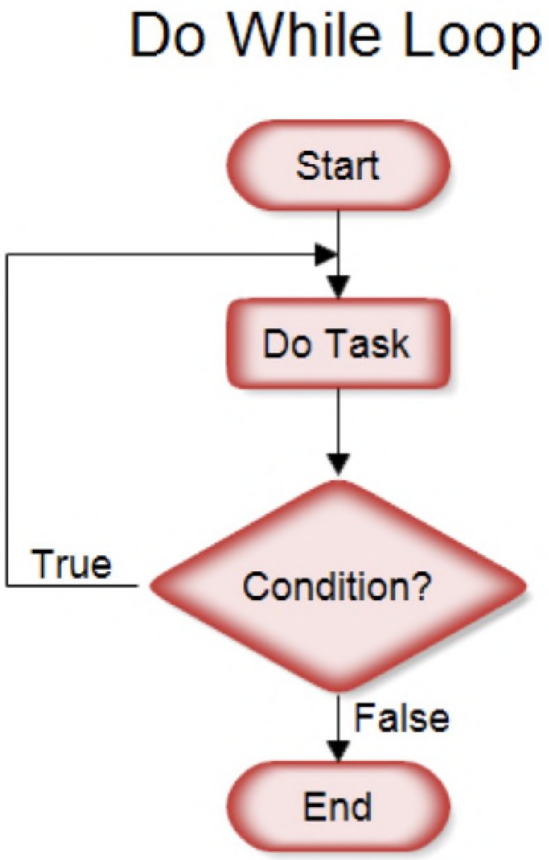
```
int sum = 0;
int i = 1;

while (i < 10)
{
    sum = sum + i;
    i += 2;
}
```

do-while statement

```
do
        statement;
```

```
while (expression);
```




```
do
{
```

Examples

What are the values of **sum** and **i** after the loop?

```
int sum = 0;
int i = 10;
do
{
        sum = sum + i;
        i += 2;

} while (i < 10);
```

break and continue statement

- break;
  - terminates loop
  - execution continues with the first statement following the loop
- continue;
  - forces next iteration of the loop, skipping any remaining statements in the loop